

```
\maximum_frequency][0]
combined_term = ' '.join(highest_frequency_term)
```

In this snippet, we started by defining `frequencies`, a list containing the frequency of every $n + 1$ -gram in our corpus that matches the search term. Then, we used the `numpy` module's `max()` function to find the highest of those frequencies. We used another list comprehension to get the first $n + 1$ -gram that occurs with the highest frequency in the corpus, and finally we created a `combined_term`, a string that puts together all of the words in that search term, with spaces separating the words.

Finally, we can put all of our code together in a function, shown in [Listing 8-2](#).

```
def search_suggestion(search_term, text):
    token = nltk.word_tokenize(text)
    bigrams = ngrams(token,2)
    trigrams = ngrams(token,3)
    fourgrams = ngrams(token,4)
    fivegrams = ngrams(token,5)
    grams = [ngrams(token,2),ngrams(token,3),ngrams(token,4),ngrams(token,5)]
    split_term = tuple(search_term.split(' '))
    search_term_length = len(search_term.split(' '))
    counted_grams = Counter(grams[search_term_length-1])
    combined_term = 'No suggested searches'
    matching_terms = [element for element in list(counted_grams.items()) if
\element[0][: -1] == tuple(split_term)]
    if(len(matching_terms) > 0):
        frequencies = [item[1] for item in matching_terms]
        maximum_frequency = np.max(frequencies)
        highest_frequency_term = [item[0] for item in matching_terms if item[1]
== \maximum_frequency][0]
        combined_term = ' '.join(highest_frequency_term)
    return(combined_term)
```

Listing 8-2: A function that provides search suggestions by taking an n -gram and returning the most likely $n + 1$ -gram that starts with the input n -gram

When we call our function, we pass an n -gram as the argument, and the function returns an $n + 1$ -gram. We call it as follows: